General Information

We start by having matrix's A B C, we allocate them in a 1D fashion, and they represent a 1D structure. To have deterministic checking we fill A and B by using the following method.

```
void fill_matrix(double *m, double *m2,int n) {
    for (int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            m[i*n + j] = sin(i) * cos(j) + sqrt(i + j + 1);
            m2[i*n + j] = cos(i) * sin(j) + sqrt(i + j + 2);
        }
    }
}
```

How we fill the matrix

The program at the most basic level provides different modes for calculating C = A x B all with the required triple loop method. In addition to having the matrix multiplication calculated we also have 3 additional parameters calculated.

- SumC, the sum of all elements in C
- MaxC, the max element in C
- Checksum, deterministic checksum

We calculate these in a serial fashion based on the state of Matrix C from a mode except for mode 3 which utilizes an atomic and critical comparison for these required calculations. Each mode can be run based on the specifications of the assignment after it is compiled. Each mode will run 11 times throwing out the first run and calculate ethe average, this average as well as other parameters like number of threads, and problem size, will be used to calculate speedup and efficiency.

Design summary

Most of the outer loop structures in this assignment were done with #pragma omp parallel for. In our case i and j looping indexes will be private to the scope of the loop and the parallel section.

For mode 1 we had to use dynamic schedule and static to complete the requirements of the assignment the dynamic contains a chunk size that is adjusted appropriately based on the size of the problem and number of threads being run. The chunk size for dynamic scheduling was adjusted based on max number of threads your machine can have, I believe this was the simplest way to scale with the problem size without having to stress about exact numbers.

When we have parallel regions that rely on chunking the problem we used sharing among the threads for the matrices and the sum portion of the region was private. For our task based problem in mode 4 we used firstprivate, so we ensure that each task in the region gets its unique copy of start and end row indicators.

In this assignment we use the checksum to make sure that threads must be synchronizing information correctly. I used the checksum formula indicated in the assignment document. In mode 3 we compare both the atomic and critical options for this checksum calculation. The atomic was used where checksum is incremented and the critical section encompasses it. The required values for sumC maxC and checksum are calculated as a result of the kernel serially except for mode 3 as discussed. In this case it would be best to use atomic for this assignment because it is just performing on the single operation so we do not need an entire critical section to achieve this.

Theory Discussion

Efficiency drops as thread count increases, if the problem size doesn't change, because there is too much overhead associated with creating the threads and synchronizing them after. We will have too many threads while the actual runtime barely drops, or even increases, so our efficiency in the calculations tank.

The atomic version was always faster because for what we are calculating the atomic a small update for maintenance while the critical section needs more overhead for each pass.

Tables

N Mode Threads Schedule Time(s) Speedup Efficiency

N	Mode	Thread	Schedule	Time	Speedup	Efficiency
512	1	2	Static	0.1214	1.93	0.96
1024	1	2	Static	1.007	1.93	0.96
2048	1	2	Static			
512	1	2	dynamic	0.1213	1.93	0.96
1024	1	2	dynamic	0.997	1.95	0.97
2048	1	2	dynamic			
512	2	2				
1024	2	2				
2048	2	2				
512	3	2				
1024	3	2				
2048	3	2				
512	4	2				
1024	4	2				
2048	4	2				
512	5	2				
1024	5	2				
2048	5	2				
512	6	2				
1024	6	2				
2048	6	2				

Tables

N Mode Threads Schedule Time(s) Speedup Efficiency

N	Mode	Thread	Schedule	Time	Speedup	Efficiency
512	1	4				
1024	1	4				
2048	1	4				
512	1	4				
1024	1	4				
2048	1	4				
512	2	4				
1024	2	4				
2048	2	4				
512	3	4				
1024	3	4				
2048	3	4				
512	4	4				
1024	4	4				
2048	4	4				
512	5	4				
1024	5	4				
2048	5	4				
512	6	4				
1024	6	4				
2048	6	4				

Tables

N Mode Threads Schedule Time(s) Speedup Efficiency

N	Mode	Thread	Schedule	Time	Speedup	Efficiency
512	1	8				
1024	1	8				
2048	1	8				
512	1	8				
1024	1	8				
2048	1	8				
512	2	8				
1024	2	8				
2048	2	8				
512	3	8				
1024	3	8				
2048	3	8				
512	4	8				
1024	4	8				
2048	4	8				
512	5	8				
1024	5	8				
2048	5	8				
512	6	8				
1024	6	8				
2048	6	8				